

Lab 7 — Movement & Networks

(NetworkX + GeoPandas + OSMnx)

Open-Source Geovisualization • GEOG 8005 / ESCI 6000 • Mar 02, 2026

Overview

In this lab you will work with three core movement-data representations: (1) networks (graphs), (2) origin-destination (OD) flows, and (3) trajectories. You will run provided demo code, then demonstrate code literacy by explaining what each block does and identifying which parameters control key visual design choices (e.g., node size, edge width, color maps, transparency, projection, log scaling, and basemaps).

Reference script: many examples in Parts 2–3 are adapted from `geo_mobility_demo.ipynb` (provided) so you can run them end-to-end.

What you will produce (portfolio-friendly)

- One non-spatial network visualization (PNG) with key nodes highlighted (degree + betweenness)
- One spatial road-network map (PNG) plus one accessibility map (route or isochrone)
- Three OD flow visuals (PNG): flow map, chord diagram, and OD matrix heatmap
- Three trajectory visuals (PNG): line plot on map, space-time cube, and stop-detection map
- Short written answers to the code-reading questions (can be included as screenshots)

Prerequisites

- A working conda environment (e.g., `geoviz`) with: `geopandas`, `pandas`, `numpy`, `matplotlib`, `networkx`
- Recommended (for spatial networks): `osmnx`, `contextily`, `pyproj`
- Recommended (for interactive OD / trajectory visuals): `plotly`
- Optional (for chord diagrams): `holoviews` + `bokeh`
- Optional (for trajectory handling): `movingpandas`

```
# In terminal (example)
conda activate geoviz
```

```
# Core
conda install -c conda-forge geopandas pandas numpy matplotlib networkx
```

```
# Spatial networks + basemaps (recommended)
conda install -c conda-forge osmnx contextily pyproj
```

```
# Interactive (recommended)
conda install -c conda-forge plotly
```

```
# Optional: chord diagram + trajectories
conda install -c conda-forge holoviews bokeh movingpandas

python -c "import geopandas, networkx; print('OK')"
```

Data for this lab

Use ONE of the following options for each part (mix-and-match is fine).

- Option A (recommended): OpenFlights (airports + routes)
- Airports + routes data are simple text files you can download and parse in Python.
- We will use them for both: (i) network metrics and (ii) OD flows.
- Option B: Your own movement data
- You may use a dataset from your course project (roads, OD table, GPS tracks), as long as you can reproduce all required outputs.

NOTE: Starting from this lab, the demo code provided will not have a decent visualization generated; you need to modify the code to generate decent figures. You need to fix the overlap, color and redundant content when needed.

Submission (screenshots only)

Submit a series of image files containing these screenshots:

1. Successful imports + package versions (geopandas, networkx; osmnx if used)
2. Non-spatial network: node/edge counts + top node by degree + top node by betweenness
3. Non-spatial network figure with highlighted nodes (degree + betweenness)
4. Road network download/preview (map)
5. Accessibility output (route or isochrone map)
6. OD checks: df.shape/dtypes + missing values + describe/hist + symmetry check + spatial coverage
7. Flow map (OD) figure
8. Chord diagram (or documented fallback) figure
9. OD matrix heatmap figure
10. Trajectory checks: traj counts + temporal gaps + speed stats + length stats + spatial coverage
11. Map matching before/after figure (if you map match)
12. Gap filling before/after figure (if you fill gaps)
13. Trajectory line plot on map figure
14. Space-time cube figure
15. Stop detection map figure
16. Optional: edge bundling (R) output

Part 1 — Networks and spatial networks

1.1 Project folders

- Keep a clean structure so your work is reproducible.

```
from pathlib import Path

DATA = Path("data/raw")
OUT  = Path("outputs")
FIG  = OUT / "figs"
for p in [DATA, OUT, FIG]:
    p.mkdir(parents=True, exist_ok=True)

print("Folders ready:", DATA, OUT, FIG)
```

1.2 Download OpenFlights (airports + routes)

- Download two files: airports.dat and routes.dat.
- If your campus network blocks downloads, download manually and place them in data/raw/.

```
import urllib.request
from pathlib import Path

airports_url =
"https://raw.githubusercontent.com/jpatokal/openflights/master/data/airports.dat"
routes_url   =
"https://raw.githubusercontent.com/jpatokal/openflights/master/data/routes.dat"

airports_path = Path("data/raw/airports.dat")
routes_path   = Path("data/raw/routes.dat")

urllib.request.urlretrieve(airports_url, airports_path)
urllib.request.urlretrieve(routes_url, routes_path)

print("Downloaded:", airports_path, routes_path)
```

1.3 Load tables + basic inspection

- Read both files into pandas and inspect columns and a few rows.

```
import pandas as pd

airports_cols =
"airport_id,name,city,country,iata,icao,lat,lon,alt,tz,dst,tz_db,type,source".split(",")
```

```

routes_cols =
"airline,airline_id,src_iata,src_id,dst_iata,dst_id,codeshare,stops,equipment".split(",")

airports = pd.read_csv("data/raw/airports.dat", header=None, names=airports_cols)
routes = pd.read_csv("data/raw/routes.dat", header=None, names=routes_cols)

print("airports:", airports.shape, "routes:", routes.shape)
print(airports[["iata","name","country","lat","lon"]].head())
print(routes[["src_iata","dst_iata","stops"]].head())

```

1.4 Clean + build a graph (NetworkX)

- Remove routes with missing IATA codes and build a weighted directed graph.
- For faster centrality, we will keep only the most-connected airports.

```

import networkx as nx

# Clean (OpenFlights uses literal string "\N" for missing codes)
for c in ["src_iata", "dst_iata"]:
    routes[c] = routes[c].astype(str)

mask = (
    routes["src_iata"].notna()
    & routes["dst_iata"].notna()
    & (routes["src_iata"] != r"\N")
    & (routes["dst_iata"] != r"\N")
    & (routes["src_iata"].str.len() > 0)
    & (routes["dst_iata"].str.len() > 0)
)

routes2 = routes.loc[mask, ["src_iata", "dst_iata"]].copy()
routes2 = routes2[routes2["src_iata"] != routes2["dst_iata"]] # drop self-loops

# Aggregate to OD counts (edge weight)
od = (
    routes2.groupby(["src_iata", "dst_iata"], as_index=False)
        .size()
        .rename(columns={"size": "flow"})
)

# Build weighted directed graph
G = nx.from_pandas_edgelist(
    od, source="src_iata", target="dst_iata",
    edge_attr="flow", create_using=nx.DiGraph()
)

print("Graph nodes:", G.number_of_nodes(), "edges:", G.number_of_edges())

# Keep a manageable subgraph for centrality (top N by degree)
deg = dict(G.degree())

```

```
top_nodes = sorted(deg, key=deg.get, reverse=True)[:200]
H = G.subgraph(top_nodes).copy()
print("Subgraph nodes:", H.number_of_nodes(), "edges:", H.number_of_edges())
```

1.5 Non-spatial network visualization (symbology controls)

- Goal: a clean network diagram where node size and edge width communicate importance.
- Save your figure to outputs/figs/network_nospatial.png.

Visualization tuning (what to modify):

- Node size: change the scaling function used for node_sizes (e.g., degree-based scaling, min/max, or log scaling).
- Node color: change node_color for default nodes and highlight nodes (e.g., top-degree in orange, top-betweenness in magenta).
- Edge width: change edge_widths scaling (e.g., based on flow, log1p(flow), or a fixed width).
- Transparency: change alpha for nodes/edges to reduce clutter in dense graphs.
- Layout: swap nx.spring_layout(...) with kamada_kawai_layout(...) or adjust spring_layout k/iterations/seed.

```
import matplotlib.pyplot as plt

pos = nx.spring_layout(H, seed=7, k=0.25)

# — Visual design knobs (try changing these) —————
node_size_scale = 40      # larger = bigger nodes
edge_width_scale = 0.12   # larger = thicker edges
node_alpha = 0.92
edge_alpha = 0.20
bg_color = "#0f1117"      # dark background; try "white" for print
node_cmap = "plasma"      # try "viridis", "cool", "YlOrRd"
edge_color = "#4a90d9"    # try "white" or "gray"
# —————

deg_H = dict(H.degree())
node_sizes = [deg_H[n] * node_size_scale for n in H.nodes()]
node_colors = [deg_H[n] for n in H.nodes()] # color encodes degree
edge_widths = [H[u][v]["flow"] * edge_width_scale for u, v in H.edges()]

fig, ax = plt.subplots(figsize=(11, 9), facecolor=bg_color)
ax.set_facecolor(bg_color)

nx.draw_networkx_edges(H, pos, ax=ax,
    width=edge_widths, alpha=edge_alpha, edge_color=edge_color)
nc = nx.draw_networkx_nodes(H, pos, ax=ax,
```

```

node_size=node_sizes, node_color=node_colors,
cmap=node_cmap, alpha=node_alpha)

sm = plt.cm.ScalarMappable(cmap=node_cmap,
    norm=plt.Normalize(min(node_colors), max(node_colors)))
plt.colorbar(sm, ax=ax, shrink=0.6, label="Degree", pad=0.01)
ax.set_title("OpenFlights network – node size & color = degree",
    color="white", fontsize=13)
ax.axis("off")

out = "outputs/figs/network_nospatial.png"
plt.tight_layout()
plt.savefig(out, dpi=200, facecolor=bg_color)
plt.show()
print("Saved:", out)

```

1.6 Highlight the highest-degree node and highest-betweenness node

- Compute two metrics and highlight the top node for each.
- Save to outputs/figs/network_highlight.png.

```

deg_H = dict(H.degree())
top_deg_node = max(deg_H, key=deg_H.get)

# Betweenness (undirected for readability)
bw = nx.betweenness centrality(H.to_undirected(), normalized=True)
top_bw_node = max(bw, key=bw.get)

print('Top degree node:', top_deg_node, 'degree:', deg_H[top_deg_node])
print('Top betweenness node:', top_bw_node, 'betweenness:', bw[top_bw_node])

# — Visual design knobs —————
bg          = '#0f1117'    # background color
color_deg   = '#FF8C00'    # orange – top degree node
color_bw    = '#FF00FF'    # magenta – top betweenness node
hl_node_size = 1800        # size of the two highlighted nodes
legend_ms   = 10           # legend marker size (pts) – keep small!
# —————

# TO DO (student exercise):
# 1. Change color_deg / color_bw to any hex color you prefer.
# 2. Try hl_node_size = 800 vs 2500 – how does it affect readability?
# 3. Change legend_ms (try 6–14) to control legend circle size.
# 4. Move the legend: loc='upper right', 'lower right', etc.
# —————

from matplotlib.lines import Line2D

```

```

fig, ax = plt.subplots(figsize=(11, 9), facecolor=bg)
ax.set_facecolor(bg)

nx.draw_networkx_edges(H, pos, ax=ax, width=edge_widths, alpha=0.18,
    edge_color='#4a90d9')
nx.draw_networkx_nodes(H, pos, ax=ax, node_size=node_sizes,
    node_color='gray', alpha=0.55)

# Highlighted nodes – NO label= here (would create oversized legend circles)
nx.draw_networkx_nodes(H, pos, ax=ax, nodelist=[top_deg_node],
    node_size=hl_node_size, node_color=color_deg,
    linewidths=3, edgecolors='white')
nx.draw_networkx_nodes(H, pos, ax=ax, nodelist=[top_bw_node],
    node_size=hl_node_size, node_color=color_bw,
    linewidths=3, edgecolors='white')

nx.draw_networkx_labels(H, pos, ax=ax,
    labels={top_deg_node: top_deg_node, top_bw_node: top_bw_node},
    font_size=11, font_color='white', font_weight='bold')

# Build legend with small fixed-size markers (not inherited from node_size)
legend_handles = [
    Line2D([0], [0], marker='o', color='none',
        markerfacecolor=color_deg, markeredgecolor='white',
        markeredgewidth=0.8, markersize=legend_ms,
        label=f'Top degree: {top_deg_node}'),
    Line2D([0], [0], marker='o', color='none',
        markerfacecolor=color_bw, markeredgecolor='white',
        markeredgewidth=0.8, markersize=legend_ms,
        label=f'Top betweenness: {top_bw_node}'),
]
ax.legend(handles=legend_handles, loc='lower left', fontsize=10,
    facecolor='#222', labelcolor='white',
    framealpha=0.85, borderpad=0.8)

ax.set_title(
    'Highlighted: top degree (orange) vs top betweenness (magenta)',
    color='white', fontsize=13)
ax.axis('off')

out = 'outputs/figs/network_highlight.png'
plt.tight_layout()
plt.savefig(out, dpi=200, facecolor=bg)
plt.show()
print('Saved:', out)

```

1.7 Download + visualize a spatial road network (OSMnx)

- Goal: download a drivable road network and map it as a spatial network.
- Save to outputs/figs/road_network.png.

Road network visualization tuning (what to modify):

- Study area: edit PLACE_CANDIDATES (or the fallback center_latlon/dist_m).
- Extent: change dist_m (fallback) or choose a smaller polygon (e.g., borough/neighborhood) to avoid huge downloads.
- Line width / readability: edit linewidth and alpha in edges_web.plot(...).
- Basemap style: in ctx.add_basemap, set source=ctx.providers.CartoDB.Positron (or another provider).

```
import os
import osmnx as ox
import contextily as ctx
import matplotlib.pyplot as plt

ox.settings.log_console = True
ox.settings.use_cache = True
ox.settings.timeout = 180

# Tip: Change PLACE to try a different city/region
PLACE_CANDIDATES = [
    "Manhattan, New York City, New York, USA",
    "New York City, New York, USA",
    "Seattle, Washington, USA",
]

network_type = "drive"

# 1) Try place -> polygon, then fallback to point+radius
polygon = None
for place in PLACE_CANDIDATES:
    try:
        gdf_place = ox.geocode_to_gdf(place)
        if len(gdf_place) > 0:
            polygon = gdf_place.geometry.iloc[0]
            print("Geocoded place OK:", place)
            break
    except Exception as e:
        print("Geocode failed:", place, "|", type(e).__name__, e)

if polygon is not None:
    G_road = ox.graph_from_polygon(polygon, network_type=network_type,
simplify=True)
else:
```



```

print("Fallback: graph_from_point (no geocode results).")
# Example point (NYC-ish). Change as needed.
center_latlon = (40.7580, -73.9855)
dist_m = 3000
G_road = ox.graph_from_point(center_latlon, dist=dist_m,
network_type=network_type, simplify=True)

# 2) Project + convert to GeoDataFrames
G_road = ox.project_graph(G_road)
nodes_gdf, edges_gdf = ox.graph_to_gdfs(G_road)
print("Road nodes:", nodes_gdf.shape, "Road edges:", edges_gdf.shape)

# 3) Plot + basemap (contextily wants Web Mercator)
edges_web = edges_gdf.to_crs(epsg=3857)

fig, ax = plt.subplots(figsize=(10, 8))
edges_web.plot(ax=ax, linewidth=0.6, alpha=0.8)

ctx.add_basemap(ax, crs=edges_web.crs)
ax.set_title("Road network (drive)")
ax.set_axis_off()

os.makedirs("outputs/figs", exist_ok=True)
out = "outputs/figs/road_network.png"
plt.tight_layout()
plt.savefig(out, dpi=200)
plt.show()
print("Saved:", out)

```

1.8 Accessibility demo (shortest path)

- Pick an origin point (e.g., campus) and compute accessibility on the road network.
- Save to outputs/figs/accessibility.png.

Accessibility tuning (what to modify):

- Origin & destination: change the origin point and the target POI category (hospital, grocery, bus stop, etc.).
- Edge weights: set weight='length' (meters) for shortest-distance, or add travel_time for shortest-time.
- Route symbology: change route line color, linewidth, and zorder for emphasis.
- Accessibility definition: replace a single shortest path with an isochrone (reachable area within X minutes).

```

import networkx as nx
import numpy as np

origin_latlon = ox.geocode(place) # (lat, lon)
orig_node = ox.nearest_nodes(G_road, X=origin_latlon[1], Y=origin_latlon[0])
print("Origin node:", orig_node)

hosp = ox.features_from_place(place, tags={"amenity":
"hospital"}).to_crs(nodes_gdf.crs)
hosp_pts = hosp.geometry.centroid
target_nodes = [ox.nearest_nodes(G_road, X=p.x, Y=p.y) for p in hosp_pts]

dists = [nx.shortest_path_length(G_road, orig_node, t, weight="length") for t in
target_nodes]
best_t = target_nodes[int(np.argmin(dists))]
print("Nearest hospital node:", best_t, "dist_m:", float(np.min(dists)))

route = nx.shortest_path(G_road, orig_node, best_t, weight="length")
route_gdf = ox.routing.route_to_gdf(G_road, route)

fig, ax = plt.subplots(figsize=(10, 8))
edges_gdf.plot(ax=ax, linewidth=0.4, alpha=0.4)
route_gdf.plot(ax=ax, linewidth=3)

ctx.add_basemap(ax, crs=edges_gdf.crs)
ax.set_title("Accessibility demo: shortest path to nearest hospital")
ax.set_axis_off()

out = "outputs/figs/accessibility.png"
plt.tight_layout()
plt.savefig(out, dpi=200)
plt.show()
print("Saved:", out)

```

Part 2 — Origin-Destination (OD) flow visualization

Check your understanding (Part 2)

Flow map tuning (what to modify):

- Filtering: change top N flows (e.g., `nlargest(200)` -> 100/300) to balance readability.
- Line width: change the scaling from `flow/log1p(flow)` to line width (min/max widths).
- Color: change line color to show direction (e.g., origin-based color) or magnitude (colormap).
- Transparency: increase/decrease alpha to reduce overplotting.

Chord diagram tuning (what to modify):

- Top K nodes: reduce/increase K to control clutter.
- Edge threshold: filter out small flows before plotting (e.g., flow ≥ 5).
- Node ordering: sort nodes by total flow to make the chord easier to read.

OD matrix tuning (what to modify):

- Log scaling: use `np.log1p(matrix)` to reveal structure in heavy-tailed flows.
- Colormap: choose a perceptually uniform colormap (e.g., 'viridis') and adjust `vmin/vmax`.
- Top K: use fewer nodes for readability and faster plotting.
- Normalization: optionally normalize rows (origin totals) to compare patterns rather than volumes.
- Colors: assign node colors by region/country, or by inbound/outbound dominance.
- Background: change basemap provider or remove basemap for a cleaner print map.

17. In `spring_layout(H, seed=7, k=0.25)`, what does `k` control? What happens if you increase it?

18. Which lines control node size, edge width, and transparency in the non-spatial network plot?

19. Why do we project the road network graph before computing distances?

20. Degree vs betweenness: what does each metric capture about a node's role in the network?

21. In OSMnx, what does `network_type='drive'` change compared to 'walk'?

2.1 Build an OD table (from routes) + join coordinates

- Aggregate routes into OD flows and attach latitude/longitude for origin and destination.
- Save the cleaned OD table to `outputs/od_flows.csv`.

```
import os
import pandas as pd

# Clean airports table (OpenFlights uses literal string "\N")
air = airports[["iata", "lat", "lon", "name", "country"]].copy()
air["iata"] = air["iata"].astype(str)

mask = (
    air["iata"].notna()
    & (air["iata"] != r"\N")
    & (air["iata"].str.len() > 0)
)
air = air.loc[mask].drop_duplicates("iata").copy()

air["lat"] = pd.to_numeric(air["lat"], errors="coerce")
air["lon"] = pd.to_numeric(air["lon"], errors="coerce")

# Join OD flows with origin/destination airport coordinates
od2 = (
    od.merge(air.add_prefix("o_"), left_on="src_iata", right_on="o_iata",
    how="left")
```

```

        .merge(air.add_prefix("d_"), left_on="dst_iata", right_on="d_iata",
how="left")
    )

od2 = od2.dropna(subset=["o_lat", "o_lon", "d_lat", "d_lon"]).copy()
print("OD rows after join:", od2.shape)

os.makedirs("outputs", exist_ok=True)
out_csv = "outputs/od_flows.csv"
od2.to_csv(out_csv, index=False)
print("Saved:", out_csv)

```

2.2 OD data checks (shape, missing, distribution, symmetry, coverage)

- Run these checks and interpret what they imply for visualization choices.

```

import numpy as np
import matplotlib.pyplot as plt

df = od2.copy()

print("df.shape:", df.shape)
print(df.dtypes)

print(df.isnull().sum())

print(df["flow"].describe())

plt.figure(figsize=(6,4))
plt.hist(df["flow"], bins=50)
plt.title("Flow distribution (raw)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.hist(np.log1p(df["flow"]), bins=50)
plt.title("Flow distribution (log1p)")
plt.tight_layout()
plt.show()

rev =
df.rename(columns={"src_iata":"dst_iata","dst_iata":"src_iata","flow":"flow_rev"})
sym = df.merge(rev[["src_iata","dst_iata","flow_rev"]],
on=["src_iata","dst_iata"], how="left")
sym["diff"] = sym["flow"] - sym["flow_rev"].fillna(0)

```



```

colors = [cmap(norm(f)) for f in gdf_lines["flow"]]

fig, ax = plt.subplots(figsize=(14, 8))
for (_, row), color in zip(gdf_lines.iterrows(), colors):
    gpd.GeoDataFrame([row], geometry="geometry",
                      crs=gdf_lines.crs).plot(
        ax=ax, linewidth=row["w_plot"], alpha=alpha, color=color)

sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
plt.colorbar(sm, ax=ax, shrink=0.5, label="Flow (routes)", pad=0.01)

ctx.add_basemap(ax, crs=gdf_lines.crs,
                source=ctx.providers.CartoDB.DarkMatter)
ax.set_title(f"OD flow map – top {N} flows (curved arcs, color = magnitude)",
             fontsize=13)
ax.set_axis_off()

out = "outputs/figs/od_flow_map.png"
plt.tight_layout()
plt.savefig(out, dpi=200)
plt.show()
print("Saved:", out)

```

2.4 Chord diagram (top airports)

- Chord diagrams get messy fast. Restrict to the top K airports by total flow.
- Recommended: holoviews (hv.Chord). If it fails, document your fallback (e.g., Sankey).
- Save to outputs/figs/od_chord.png (export/screenshot).

```

# Option A: holoviews chord (interactive in Jupyter)
import pandas as pd
import holoviews as hv
from holoviews import opts
hv.extension("bokeh")

K = 20
topK = (df.groupby("src_iata")["flow"].sum()
        .sort_values(ascending=False)
        .head(K)
        .index)

dfK = df[df["src_iata"].isin(topK) & df["dst_iata"].isin(topK)].copy()

edges = dfK[["src_iata", "dst_iata", "flow"]]
nodes = pd.DataFrame({"iata":
    sorted(set(edges["src_iata"]).union(edges["dst_iata"]))})

```

```

ch = hv.Chord((edges, hv.Dataset(nodes, "iata"))).opts(
    opts.Chord(labels="iata", edge_color="src_iata", node_color="iata",
               edge_alpha=0.6, node_size=10, width=650, height=650,
               title=f"Chord diagram (top {K} airports)")
)
ch

```

2.5 OD matrix (heatmap)

- Create an OD matrix for the top K airports and visualize it as a heatmap.
- Save to outputs/figs/od_matrix.png.

```

import seaborn as sns

K = 25
topK = (df.groupby("src_iata")["flow"].sum()
        .sort_values(ascending=False).head(K).index)

dfK = df[df["src_iata"].isin(topK) & df["dst_iata"].isin(topK)].copy()

mat = (dfK.pivot_table(index="src_iata", columns="dst_iata",
                       values="flow", aggfunc="sum", fill_value=0)
       .reindex(index=topK, columns=topK, fill_value=0))

# — Visual design knobs —————
cmap_name = "magma"      # try "YlOrRd", "Blues", "RdYlGn_r"
use_log    = True        # log-transform to reveal structure in heavy tails
# —————

mat_plot = np.log1p(mat) if use_log else mat
cb_label  = "log(1+flow)" if use_log else "flow"

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(mat_plot, ax=ax,
            cmap=cmap_name,
            linewidths=0.3, linecolor="gray",
            square=True,
            xticklabels=True, yticklabels=True,
            cbar_kws={"label": cb_label, "shrink": 0.75})

ax.set_title(f"OD matrix – top {K} airports by total flow", fontsize=14)
ax.set_xlabel("Destination IATA", fontsize=11)
ax.set_ylabel("Origin IATA",      fontsize=11)
ax.tick_params(axis="x", rotation=90, labels=8)
ax.tick_params(axis="y", rotation=0,  labels=8)

```

```
out = "outputs/figs/od_matrix.png"
plt.tight_layout()
plt.savefig(out, dpi=200)
plt.show()
print("Saved:", out)
```

Check your understanding (Part 2)

22. Why do we often use $\log_{1p}(\text{flow})$ for flow maps and OD matrices?
23. Which checks tell you whether your OD table is sparse or dense?
24. How did we test symmetry (A→B vs B→A)? What does non-symmetry mean for interpretation?
25. In the flow map, which parameters control line width and transparency?
26. Why do chord diagrams usually require filtering to top K nodes?

Part 3 — Trajectories (GPS tracks)

3.1 Load trajectory data

- Use a trajectory table with at least: traj_id, t (timestamp), lat, lon.
- If you do not have a dataset, use a provided sample from the course site (or generate synthetic data).

Use the following code to download a small real trajectory dataset (recommended), with a synthetic fallback if download fails.

```
import os
import io, requests
import pandas as pd
import geopandas as gpd
from pathlib import Path

import os, io
import pandas as pd
import geopandas as gpd
import numpy as np
import networkx as nx
from pathlib import Path
from shapely.geometry import Point, LineString

# -----
# Road-following synthetic GPS trajectories WITH stop events
# -----
# REQUIRES: G_road, nodes_gdf, edges_gdf from Section 1.7.
# -----
```



```

rng = np.random.default_rng(42)

N_TRAJ          = 6      # number of synthetic vehicle trajectories
PING_INTERVAL_S = 15     # nominal GPS ping interval (seconds)
GPS_NOISE_M      = 8      # +/-8 m GPS noise
GAP_PROB         = 0.08   # fraction of pings randomly dropped
STOP_PROB        = 0.50   # prob of stopping at each intersection node
MIN_STOPS        = 2      # guarantee at least this many stops per route
STOP_DUR_RANGE   = (45, 150) # stop duration range in seconds

def sample_route_pings(G, nodes_gdf, src, dst,
                      ping_s, noise_m, gap_p, stop_p, min_s, stop_dur,
                      rng, tid, base_t):
    try:
        path_nodes = nx.shortest_path(G, src, dst, weight='length')
    except nx.NetworkXNoPath:
        return []
    if len(path_nodes) < 3:
        return []

    # Build full polyline and record cumulative dist at each intermediate node
    coords = []
    node_dists = [0.0]
    cumlen = 0.0
    for u, v in zip(path_nodes[:-1], path_nodes[1:]):
        ed = G[u][v]
        if isinstance(ed, dict) and 0 in ed:
            ed = ed[0]
        geom = ed.get('geometry', None)
        if geom is not None:
            seg = list(geom.coords)
            coords.extend(seg)
            cumlen += LineString(seg).length
        else:
            p1 = (nodes_gdf.loc[u].geometry.x, nodes_gdf.loc[u].geometry.y)
            p2 = (nodes_gdf.loc[v].geometry.x, nodes_gdf.loc[v].geometry.y)
            coords.extend([p1, p2])
            cumlen += np.hypot(p2[0]-p1[0], p2[1]-p1[1])
        node_dists.append(cumlen)

    route_line = LineString(coords)
    total_len = route_line.length
    mid_dists = node_dists[1:-1] # intermediate nodes only

    # Stochastic stop selection, then guarantee MIN_STOPS
    stop_mask = [rng.random() < stop_p for _ in mid_dists]
    if mid_dists and sum(stop_mask) < min_s:

```

```

    # force-enable random nodes until we reach MIN_STOPS
    off_idx = [i for i, m in enumerate(stop_mask) if not m]
    rng.shuffle(off_idx)
    for i in off_idx[:max(0, min_s - sum(stop_mask))]:
        stop_mask[i] = True
    stop_events = {mid_dists[i]: int(rng.integers(stop_dur[0], stop_dur[1]))
                   for i, m in enumerate(stop_mask) if m}

    speed_ms = rng.uniform(15, 30) / 3.6
    step      = speed_ms * ping_s

    rows = []
    dist = 0.0
    t_sec = 0.0
    fired = set() # avoid firing the same stop twice

    while dist < total_len:
        # Inject stop if within half a step of a stop node
        for stop_dist, stop_secs in stop_events.items():
            if stop_dist not in fired and abs(dist - stop_dist) < step * 0.55:
                fired.add(stop_dist)
                n_sp = max(1, stop_secs // ping_s)
                pt_s = route_line.interpolate(stop_dist)
                gs_s = gpd.GeoSeries([pt_s], crs=nodes_gdf.crs).to_crs('EPSG:4326')
                slon0, slat0 = gs_s.iloc[0].x, gs_s.iloc[0].y
                nd = noise_m * 1e-5
                for _ in range(n_sp):
                    if rng.random() > gap_p:
                        t = base_t + pd.Timedelta(seconds=t_sec)
                        rows.append((f'T{tid:02d}', t,
                                    slat0 + rng.normal(0, nd * 0.15),
                                    slon0 + rng.normal(0, nd * 0.15)))
                    t_sec += ping_s
                break

        # Normal moving ping
        pt = route_line.interpolate(dist)
        gs = gpd.GeoSeries([pt], crs=nodes_gdf.crs).to_crs('EPSG:4326')
        lon0, lat0 = gs.iloc[0].x, gs.iloc[0].y
        nd = noise_m * 1e-5
        if rng.random() > gap_p:
            t = base_t + pd.Timedelta(seconds=t_sec)
            rows.append((f'T{tid:02d}', t,
                        lat0 + rng.normal(0, nd),
                        lon0 + rng.normal(0, nd)))

        dist += step
        t_sec += ping_s + int(rng.integers(0, 5))

```

```

        return rows

all_nodes = list(G_road.nodes())
rows_all = []
base_t = pd.Timestamp('2024-06-15 08:00:00')

for tid in range(N_TRAJ):
    for _ in range(60):
        src, dst = rng.choice(all_nodes, size=2, replace=False)
        try:
            plen = nx.shortest_path_length(G_road, src, dst, weight='length')
        except nx.NetworkXNoPath:
            continue
        if plen > 500:
            break
    offset = pd.Timedelta(minutes=int(rng.integers(0, 20)))
    rows_all.extend(sample_route_pings(
        G_road, nodes_gdf, src, dst,
        PING_INTERVAL_S, GPS_NOISE_M, GAP_PROB,
        STOP_PROB, MIN_STOPS, STOP_DUR_RANGE,
        rng, tid, base_t + offset))

df = pd.DataFrame(rows_all, columns=['traj_id', 't', 'lat', 'lon'])
print(f'Generated {df["traj_id"].nunique()} trajectories, {len(df)} pings')
print(f' noise: {GPS_NOISE_M}m | interval: {PING_INTERVAL_S}s |',
      f'gap: {GAP_PROB:.0%} | stop_prob: {STOP_PROB:.0%} | min_stops: {MIN_STOPS}')
df.head()

```

```

import geopandas as gpd

# df is your road-following GPS ping table for the rest of Part 3
df_traj = df.copy()
df_traj['t'] = pd.to_datetime(df_traj['t'])

print(df_traj.head())
print('traj_id unique:', df_traj['traj_id'].nunique())
print('rows:', df_traj.shape[0])

```

3.2 Trajectory checks (count, gaps, speeds, lengths, coverage)

- Compute basic diagnostics before any map matching or smoothing.

```

import numpy as np

```

```

df_traj = df_traj.sort_values(["traj_id", "t"]).copy()

df_traj["dt_s"] = df_traj.groupby("traj_id")["t"].diff().dt.total_seconds()
print("Temporal gaps (seconds) summary:")
print(df_traj["dt_s"].describe())

def haversine_m(lon1, lat1, lon2, lat2):
    R = 6371000
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 2*R*np.arcsin(np.sqrt(a))

df_traj["lon_prev"] = df_traj.groupby("traj_id")["lon"].shift()
df_traj["lat_prev"] = df_traj.groupby("traj_id")["lat"].shift()

df_traj["dist_m"] = haversine_m(df_traj["lon_prev"], df_traj["lat_prev"],
df_traj["lon"], df_traj["lat"])
df_traj["speed_kmh"] = (df_traj["dist_m"] / df_traj["dt_s"]) * 3.6

print("Speed (km/h) summary:")
print(df_traj["speed_kmh"].describe())

by = df_traj.groupby("traj_id").agg(
    n_points=("t", "size"),
    duration_min=("dt_s", lambda s: np.nansum(s)/60),
    dist_km=("dist_m", lambda s: np.nansum(s)/1000)
)
print(by.describe())

print("Lat range:", (df_traj["lat"].min(), df_traj["lat"].max()))
print("Lon range:", (df_traj["lon"].min(), df_traj["lon"].max()))

```

3.3 Map matching (snap to road network) — before/after

- Map matching reduces GPS noise by aligning points to the road network.
- Make a before/after figure and save it to outputs/figs/mapmatch_before_after.png.

```

import geopandas as gpd
from shapely.geometry import Point, LineString
import contextily as ctx
import matplotlib.patches as mpatches

one_id = df_traj["traj_id"].iloc[0]

```

```

t0 = df_traj[df_traj["traj_id"] == one_id].copy()

gpts = gpd.GeoDataFrame(t0,
    geometry=[Point(xy) for xy in zip(t0.lon, t0.lat)],
    crs="EPSG:4326").to_crs(edges_gdf.crs)

snapped_nodes = [ox.nearest_nodes(G_road, X=pt.x, Y=pt.y)
    for pt in gpts.geometry]
snapped_xy = [(nodes_gdf.loc[n].geometry.x,
    nodes_gdf.loc[n].geometry.y)
    for n in snapped_nodes]

raw_line = LineString(list(gpts.geometry))
snap_line = LineString(snapped_xy)

graw = gpd.GeoSeries([raw_line], crs=edges_gdf.crs)
gsnap = gpd.GeoSeries([snap_line], crs=edges_gdf.crs)

# — Side-by-side panels: same extent for direct comparison —————
fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharey=True)

for ax, gline, color, title in zip(
    axes,
    [graw, gsnap],
    ["#FF4136", "#2ECC40"], # red = raw, green = snapped
    ["Before map matching\n(raw GPS trace, may drift off road)",
     "After map matching\n(snapped to road network)"]):
    edges_gdf.plot(ax=ax, linewidth=0.5, alpha=0.45, color="#888")
    gline.plot(ax=ax, linewidth=3.5, alpha=0.9, color=color)
    if color == "#FF4136": # show GPS pings on "before" panel only
        gpts.plot(ax=ax, markersize=5, color="orange",
            alpha=0.8, zorder=5)
    ctx.add_basemap(ax, crs=edges_gdf.crs,
        source=ctx.providers.CartoDB.Positron)
    ax.set_title(title, fontsize=12, pad=8)
    ax.set_axis_off()

# Lock both panels to same extent (raw trace)
xmin, ymin, xmax, ymax = graw.total_bounds
pad = (xmax - xmin) * 0.15
for ax in axes:
    ax.set_xlim(xmin - pad, xmax + pad)
    ax.set_ylim(ymin - pad, ymax + pad)

patches = [
    mpatches.Patch(color="#FF4136", label="Raw GPS path"),
    mpatches.Patch(color="orange", label="GPS pings"),

```

```

    mpatches.Patch(color="#2ECC40", label="Map-matched path"),
]
axes[1].legend(handles=patches, loc="lower right", fontsize=9)
fig.suptitle(
    "Map matching – same trajectory, same map extent for direct comparison",
    fontsize=13, y=1.01)

out = "outputs/figs/mapmatch_before_after.png"
plt.tight_layout()
plt.savefig(out, dpi=200, bbox_inches="tight")
plt.show()
print("Saved:", out)

```

3.4 Gap filling (optional) — before/after

- If `dt_s` has large gaps, you can resample to a regular interval and interpolate.
- Make a before/after plot and save to `outputs/figs/gapfill_before_after.png` (if applied).

```

# Resample one trajectory to 10-second intervals (example)
t0i = t0.set_index("t").sort_index()

interval = "10S"    # try 5S, 10S, 30S
t_rs = t0i.resample(interval).mean(numeric_only=True)
t_rs[["lat", "lon"]] = t_rs[["lat", "lon"]].interpolate()
t_rs = t_rs.reset_index()

print("Before rows:", t0i.shape[0], "After rows:", t_rs.shape[0])

# — 2x2 panel: rug plots + gap-size histograms —————
fig, axes = plt.subplots(2, 2, figsize=(16, 8),
    gridspec_kw={"hspace": 0.45, "wspace": 0.3})

before_times = t0i.index.to_series()
gaps_before = before_times.diff().dt.total_seconds().dropna()
after_idx = pd.to_datetime(t_rs["t"])
gaps_after = after_idx.diff().dt.total_seconds().dropna()

# Top-left: ping timeline BEFORE
ax = axes[0, 0]
ax.vlines(before_times, 0, 1, lw=0.8, alpha=0.7, color="#E74C3C")
ax.set_title("Before: raw ping times (irregular spacing)", fontsize=11)
ax.set_xlabel("Time"); ax.set_yticks([])
ax.tick_params(axis="x", rotation=25)

# Top-right: ping timeline AFTER
ax = axes[0, 1]

```

```

ax.vlines(after_idx, 0, 1, lw=0.8, alpha=0.7, color="#27AE60")
ax.set_title(f"After: resampled at {interval} (uniform spacing)", fontsize=11)
ax.set_xlabel("Time"); ax.set_yticks([])
ax.tick_params(axis="x", rotation=25)

# Bottom-left: gap histogram BEFORE
ax = axes[1, 0]
ax.hist(gaps_before, bins=30, color="#E74C3C", alpha=0.85, edgecolor="white")
ax.axvline(gaps_before.median(), color="black", lw=1.5, ls="--",
          label=f"Median = {gaps_before.median():.0f}s")
ax.set_title("Gap-size distribution (before) – irregular gaps", fontsize=11)
ax.set_xlabel("Gap (seconds)"); ax.set_ylabel("Count")
ax.legend(fontsize=9)

# Bottom-right: gap histogram AFTER
ax = axes[1, 1]
ax.hist(gaps_after, bins=15, color="#27AE60", alpha=0.85, edgecolor="white")
ax.axvline(gaps_after.median(), color="black", lw=1.5, ls="--",
          label=f"Median = {gaps_after.median():.0f}s")
ax.set_title(f"Gap-size distribution (after, {interval}) – all gaps equal",
          fontsize=11)
ax.set_xlabel("Gap (seconds)"); ax.set_ylabel("Count")
ax.legend(fontsize=9)

fig.suptitle(
    "Gap filling: temporal regularity before vs after resampling",
    fontsize=13, y=1.02)

out = "outputs/figs/gapfill_before_after.png"
plt.tight_layout()
plt.savefig(out, dpi=200, bbox_inches="tight")
plt.show()
print("Saved:", out)

```

3.5 Trajectory visualizations (3 forms)

- Create and save three outputs: line plot on map, space-time cube, and stop detection map.

Trajectory visualization tuning (what to modify):

- Line plot on map: change linewidth, alpha, and color; plot only a subset of trajectories if cluttered.
- Space-time cube: z-axis is time; change time units (seconds -> minutes/hours) and reduce `n_show` trajectories.
- Stop detection: adjust `min_duration` and `max_diameter` to change what counts as a stop.
- Speed coloring: clip speeds and change the colormap to emphasize slow vs fast segments.

```

# 1) Trajectory line plot – segments colored by speed
# Save: outputs/figs/trajectory_line.png
from matplotlib.collections import LineCollection
import matplotlib.colors as mcolors

t0s = t0.sort_values("t").copy()
t0s["dt_s"] = t0s["t"].diff().dt.total_seconds()
t0s["lon_prev"] = t0s["lon"].shift()
t0s["lat_prev"] = t0s["lat"].shift()
t0s["dist_m"] = haversine_m(t0s["lon_prev"], t0s["lat_prev"],
                             t0s["lon"], t0s["lat"])
t0s["speed_kmh"] = (t0s["dist_m"] / t0s["dt_s"]) * 3.6
t0s = t0s.dropna(subset=["speed_kmh"])

gpt_all = gpd.GeoDataFrame(t0s,
    geometry=[Point(xy) for xy in zip(t0s["lon"], t0s["lat"])],
    crs="EPSG:4326").to_crs(edges_gdf.crs)
xs, ys = gpt_all.geometry.x.values, gpt_all.geometry.y.values
speeds = t0s["speed_kmh"].values

segments = [(xs[i], ys[i]), (xs[i+1], ys[i+1])] for i in range(len(xs)-1)
seg_speeds = speeds[1:]

# — Visual design knobs —————
cmap_name = "RdYlGn_r" # red=fast, green=slow; try "plasma", "cool"
speed_vmax = np.percentile(seg_speeds, 95)
lw = 2.5
# —————

cmap = plt.get_cmap(cmap_name)
norm = mcolors.Normalize(vmin=0, vmax=speed_vmax)
lc = LineCollection(segments, cmap=cmap, norm=norm,
                    linewidth=lw, alpha=0.92, zorder=4)
lc.set_array(seg_speeds)

fig, ax = plt.subplots(figsize=(10, 8))
edges_gdf.plot(ax=ax, linewidth=0.4, alpha=0.35, color="#aaa")
ax.add_collection(lc)
ax.autoscale()
plt.colorbar(lc, ax=ax, shrink=0.5, label="Speed (km/h)", pad=0.01)

ctx.add_basemap(ax, crs=edges_gdf.crs,
    source=ctx.providers.CartoDB.Positron)
ax.set_title(
    "Trajectory line plot – color encodes speed (green = slow, red = fast)")
ax.set_axis_off()

```



```

out = "outputs/figs/trajectory_line.png"
plt.tight_layout()
plt.savefig(out, dpi=200)
plt.show()
print("Saved:", out)

```

```

# 2) Space-time cube (Plotly 3D) – trajectory tube colored by time
# Save: screenshot to outputs/figs/space_time_cube.png
import plotly.graph_objects as go

gpts_m = gpd.GeoDataFrame(t0s,
    geometry=[Point(xy) for xy in zip(t0s["lon"], t0s["lat"])],
    crs="EPSG:4326").to_crs(edges_gdf.crs)
gpts_m["x"] = gpts_m.geometry.x
gpts_m["y"] = gpts_m.geometry.y
gpts_m["t_min"] = (pd.to_datetime(gpts_m["t"]) -
    pd.to_datetime(gpts_m["t"]).min()).dt.total_seconds() / 60

# — Visual design knobs —————
colorscale = "Plasma" # try "Viridis", "Turbo", "RdYlGn"
line_width = 4
marker_size = 3
# —————

fig = go.Figure()
fig.add_trace(go.Scatter3d(
    x=gpts_m["x"], y=gpts_m["y"], z=gpts_m["t_min"],
    mode="lines+markers",
    line=dict(width=line_width,
        color=gpts_m["t_min"], colorscale=colorscale),
    marker=dict(size=marker_size,
        color=gpts_m["t_min"], colorscale=colorscale,
        showscale=True,
        colorbar=dict(title="Time (min)", thickness=15)),
    name="Trajectory"
))

bg = "#0f1117"
fig.update_layout(
    title="Space-time cube – trajectory tube colored by elapsed time",
    scene=dict(
        xaxis_title="Easting (m)",
        yaxis_title="Northing (m)",
        zaxis_title="Time (minutes)",
        bgcolor=bg,
        xaxis=dict(backgroundcolor=bg, gridcolor="#333"),

```

```

        yaxis=dict(backgroundcolor=bg, gridcolor="#333"),
        zaxis=dict(backgroundcolor=bg, gridcolor="#333"),
    ),
    paper_bgcolor=bg,
    font=dict(color="white"),
    margin=dict(l=0, r=0, b=0, t=40)
)
fig.show()

```

```

# 3) Stop detection -- stops sized by duration, colored by speed
# Save: outputs/figs/stop_detection.png

```

```

import numpy as np
import pandas as pd
import geopandas as gpd
from shapely.geometry import Point

```

```

# Assumes:
# - t0 has columns: ['t','lon','lat'] (and maybe others)
# - haversine_m(lon1, lat1, lon2, lat2) is defined
# - edges_gdf exists and has a projected CRS (e.g., EPSG:3857)
# - PING_INTERVAL_S is defined (fallback seconds for first point)
# - speed_thr is defined (km/h threshold)

```

```

# 1) Compute time delta, distance, speed
t0_speed = t0.sort_values("t").copy()
t0_speed["t"] = pd.to_datetime(t0_speed["t"], errors="coerce")

```

```

t0_speed["dt_s"] = t0_speed["t"].diff().dt.total_seconds()
t0_speed["lon_prev"] = t0_speed["lon"].shift()
t0_speed["lat_prev"] = t0_speed["lat"].shift()

```

```

t0_speed["dist_m"] = haversine_m(
    t0_speed["lon_prev"], t0_speed["lat_prev"],
    t0_speed["lon"],      t0_speed["lat"]
)

```

```

t0_speed["speed_kmh"] = (t0_speed["dist_m"] / t0_speed["dt_s"].replace(0, np.nan))
* 3.6

```

```

# 2) Filter slow/stop points
slow_pts = t0_speed[
    (t0_speed["speed_kmh"] < speed_thr) | (t0_speed["speed_kmh"].isna())
].copy()

```

```

# 3) Define stop duration per point (seconds)

```

```

slow_pts["stop_dur_s"] = slow_pts["dt_s"].fillna(PING_INTERVAL_S).clip(lower=0)

# 4) Create GeoDataFrame in WGS84 then project to match edges_gdf CRS
gslow = gpd.GeoDataFrame(
    slow_pts,
    geometry=gpd.points_from_xy(slow_pts["lon"], slow_pts["lat"]),
    crs="EPSG:4326"
).to_crs(edges_gdf.crs)

print("gslow created:", gslow.shape)
print("CRS:", gslow.crs)
print(gslow[["t", "lon", "lat", "dt_s", "dist_m", "speed_kmh",
"stop_dur_s"]].head())


import numpy as np
import mapclassify as mc
from matplotlib.lines import Line2D

# --- Create ONE figure/axes and keep using them ---
fig, ax = plt.subplots(figsize=(10, 8))

# background layers (example)
edges_gdf.plot(ax=ax, linewidth=0.4, alpha=0.35, color="#aaa")
gline_one = gpd.GeoSeries([raw_line], crs=edges_gdf.crs)
gline_one.plot(ax=ax, linewidth=2, alpha=0.4, color="#555", zorder=3)

if len(gslow) > 0:
    # --- Jenks (Natural Breaks) on duration ---
    k = 4
    dur = gslow["stop_dur_s"].to_numpy()
    k_eff = int(min(k, len(dur), len(np.unique(dur))))
    k_eff = max(k_eff, 2)

    nb = mc.NaturalBreaks(dur, k=k_eff)
    gslow["dur_class"] = nb.yb

    # --- Discrete sizes for classes (pt^2) ---
    size_levels = np.array([60, 160, 320, 520, 760][:k_eff]) # stronger contrast
    gslow["markersize"] = size_levels[gslow["dur_class"].values]

    # --- Scatter ---
    sc = ax.scatter(
        gslow.geometry.x, gslow.geometry.y,

```

```

s=gslow["markersize"],
c=gslow["speed_kmh"].fillna(0),
cmap=cmap_name,
alpha=0.85, zorder=6,
vmin=0, vmax=speed_thr,
edgecolors="white", linewidths=0.6
)

# Attach colorbar to
cb = fig.colorbar(sc, ax=ax, shrink=0.5, pad=0.01)
cb.set_label("Speed at stop (km/h)")

# --- Neat legend consistent with size_levels ---
bins_hi = nb.bins
bins_lo = np.r_[0, bins_hi[:-1]]

def fmt_seconds_range(a, b):
    if b < 60:
        return f"{int(a)}--{int(b)} s"
    if a < 60 <= b:
        return f"{int(a)} s - {int(round(b/60))} min"
    return f"{int(round(a/60))}--{int(round(b/60))} min"

handles = []
for i in range(k_eff):
    ms_pts = float(np.sqrt(size_levels[i])) # convert area -> points
    label = fmt_seconds_range(bins_lo[i], bins_hi[i])
    handles.append(
        Line2D([0], [0], marker="o", linestyle="",
               markersize=ms_pts,
               markerfacecolor="#cc2200",
               markeredgecolor="white",
               markeredgewidth=0.8,
               alpha=0.85,
               label=label)
    )

ax.legend(
    handles=handles,
    title="Stop duration (Jenks classes)",
    loc="lower left",
    frameon=True,
    framealpha=0.92,
    borderpad=0.8,
    labelspacing=0.7,
    handletextpad=0.8,
    fontsize=9,

```

```

        title_fontsize=9
    )

else:
    ax.text(0.5, 0.5, "No stops detected", transform=ax.transAxes,
           ha="center", va="center", fontsize=13, color="red")

# basemap + final formatting
ctx.add_basemap(ax, crs=edges_gdf.crs, source=ctx.providers.CartoDB.Positron)
ax.set_axis_off()
ax.set_title(f"Stop detection (speed < {speed_thr} km/h)\n"
            "Circle size = duration class (Jenks) | Color = speed at stop")

out = "outputs/figs/stop_detection.png"
fig.tight_layout()
fig.savefig(out, dpi=200)
plt.show()
print("Saved:", out)

```

3.6 Optional: edge bundling in R (extra credit)

- If you want a polished OD visual, try edge bundling in R using ggraph.

```

# R (example idea)
# install.packages(c("tidyverse", "sf", "tidygraph", "ggraph"))

library(tidyverse)
library(tidygraph)
library(ggraph)

# edges: src, dst, flow
# nodes: id, group (or coordinates)

# graph <- tbl_graph(nodes = nodes, edges = edges, directed = TRUE)

# ggraph(graph, layout = "circle") +
#   geom_edge_bundle(aes(width = flow), alpha = 0.4) +
#   geom_node_point(size = 2) +
#   theme_void()

```

Check your understanding (Part 3)

27. What signals in the diagnostics suggest you need map matching?
28. How is speed computed from pings, and what causes unrealistic speeds?
29. What does resampling (e.g., '10S') do, and what are the risks of interpolating?

30. In the stop detection example, which parameter(s) control what counts as a stop?
31. For the space-time cube, what does z represent and why did we convert time to seconds?